

Project : DSM

Kimia KHADEMLOU

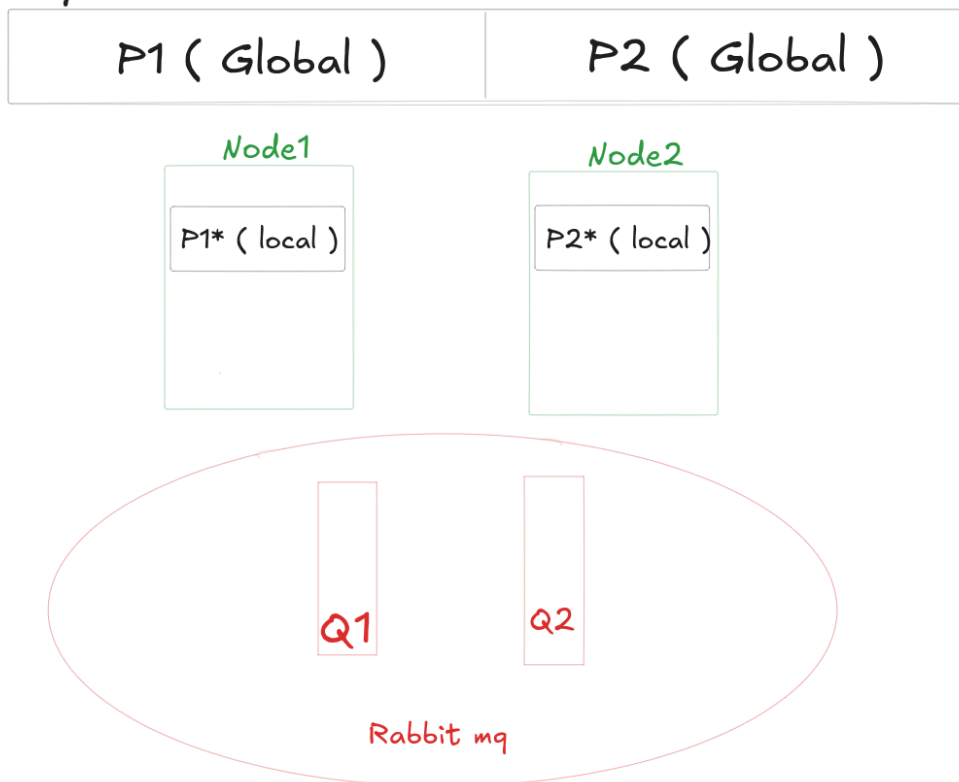
Viktoriiia Shtanko

Abdel Kader EL HOUSSAMI

M1 MOSIG

1- Design choices

Memory



This project implements a distributed shared memory system in Java where multiple nodes share a contiguous address space by message passing through RabbitMQ. This design is based upon partitioning.

- **Address model**

The global address space is split into contiguous chunks of size = $\lceil \frac{globalAddressesCnt}{(double) totalNodes} \rceil$.

Each node owns the range [startAddressGlobal ; endAddressGlobal] which corresponds to [nodeId * chunkSize ; min(startAddressGlobal + chunkSize - 1, globalAddressesCnt - 1)]. Therefore, the determination of ownership is basically an arithmetic operation.

- **Communication**

Nodes communicate through RabbitMQ. Each node has a dedicated queue named "node" + Node's ID. To send a message, a node publishes to the target's queue. To receive a message, it consumes from its own queue via a DeliverCallback. Messages are sent as arrays of bytes, and have a type that depends on the intended request : *READ_REQUEST* , *READ_RESPONSE* , *WRITE_REQUEST* , *WRITE_RESPONSE*. Furthermore, each message has a sender ID, a target ID, an address within the owner's chunk and a request ID, as well as a value and an errorMessage. Messages are used whenever a node wants to access an address that doesn't belong to its chunk. In this case, the node will determine the address' owner then send a message via RabbitMQ.

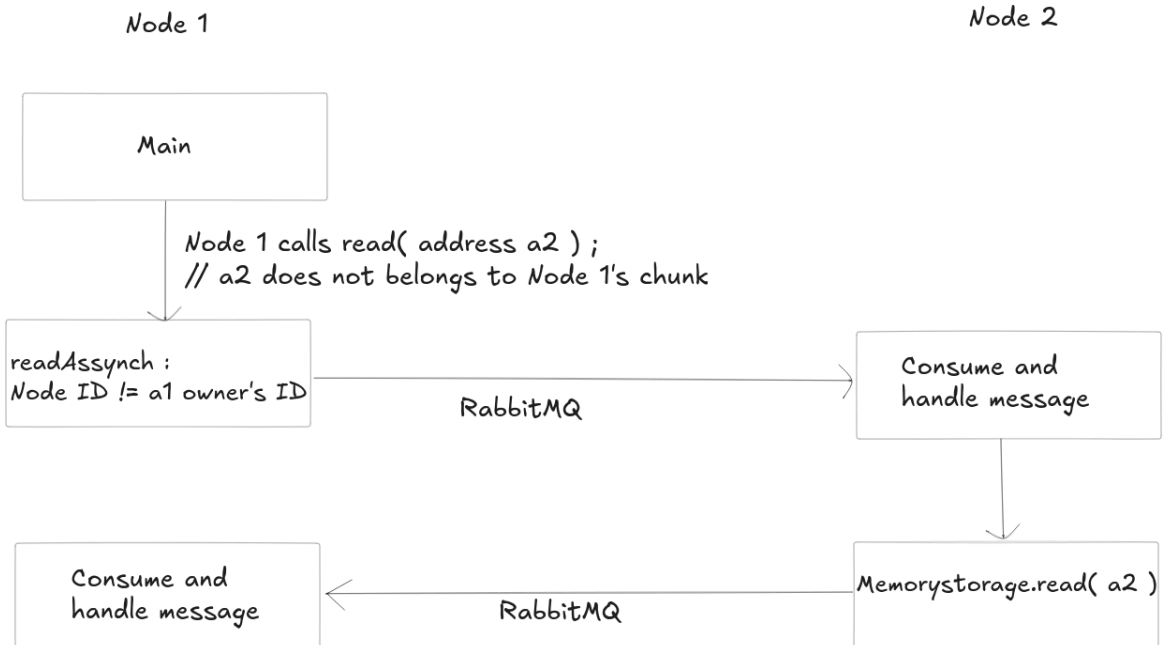
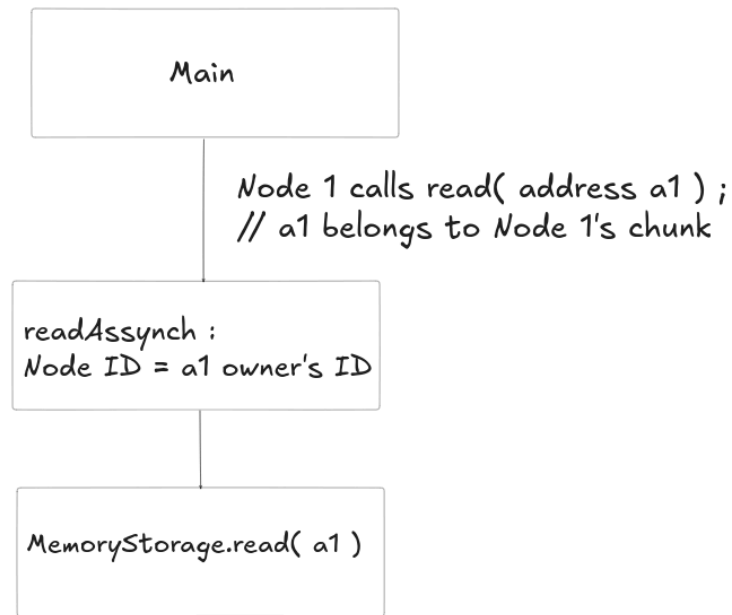
2- Implementation details

- **Asynchronous write and read**

Whenever a node needs to access a certain address 'a', the following procedure comes into execution :

1. The node's main program calls readAsync/ writeAsync.
2. The called function compares the node's ID to the address owner's ID.
 - If node's ID = address owner's : Access to a, is a local access.
 - Otherwise, a remote access is needed :
The node sends a message, via RabbitMQ, to the target node (the address' owner). The target node will receive the message and handle it. The response will be sent via RabbitMQ as well to the node that made the initial request.

We have below the two examples of memory access:



- **API Design**

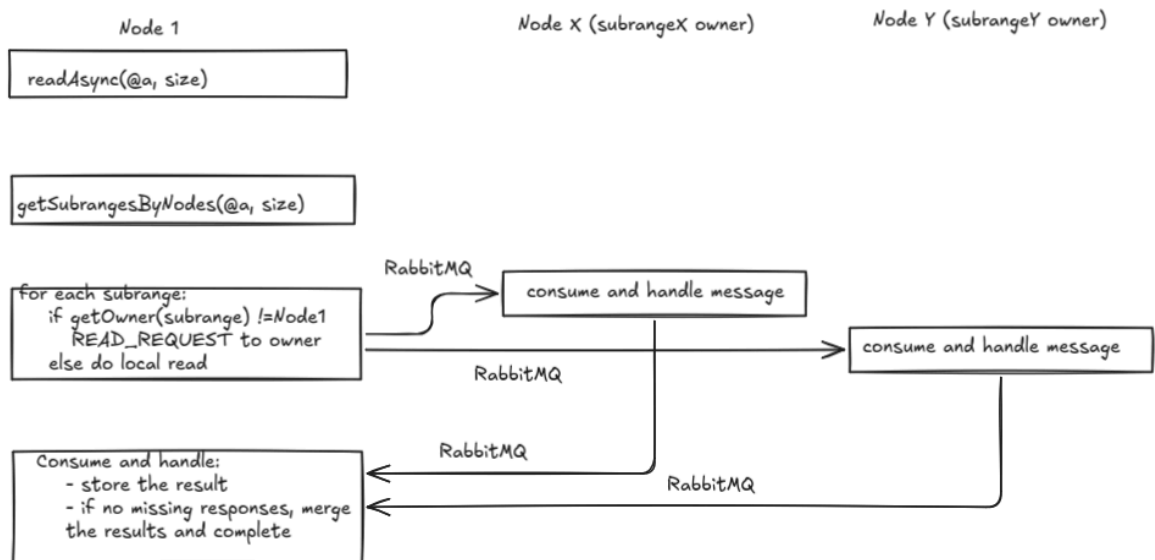
The node class reveals 2 levels of the API. The asynchronous methods return a `CompletableFuture`, an object representing a value that will arrive later. This object allows the user to call on callbacks such as *thenRun* and *thenAccept*. These callbacks will work automatically when those values arrive. Therefore, multiple reads and writes can be called at the same time, so they handle results independently.

The blocking methods read and write call *readAsync(address)* and *writeAsync(address)* and wait for their results before returning.

3- Possible extensions

- Range requests: Adding methods for reading and write address of the form *readAsync(address, size)*; *writeAsync(address, size, values)*; would make the DSM API more flexible and interesting for most applications. A proper design could leverage the sequential location of neighboring addresses in the local memory of the DSM nodes, and avoid redundant round-trips for reading ranges. The range read procedure could be as the following:

- Call to *readAsync(address, size)* is made on Node1;
- Node1 splits the range $[address, address + size - 1]$ among K nodes, each of which locally stores one chunk of the requests range;
- K subrange `READ_REQUEST` messages are sent to these nodes (as RabbitMQ messages to the corresponding queues);
- Each of the K target nodes reads locally the subrange, and return the result to the Node1 queue as a `READ_RESPONSE`;
- The request sender (Node1) waits until all the K responses have been received, and then merges them.



- Isolating client from DSM nodes: Applications running on remote servers not included in the DSM cluster might still need to access a shared memory pool over the network. This is particularly common in disaggregated data centers, where *computing* nodes are equipped with little DRAM, and have to make use of *memory* nodes. One possible design is to provide to each DSM client its own client-response queue, as well as a list of the names of queues of each DSM node. Upon a memory read request, the client can pick one of the DSM queues uniformly randomly. The receiver node will then be able either to process the request locally, or redirect it to the real owner of the requested address.
- Caching: Every remote access might be expensive. A local cache would reduce latency (partial replication).